AutoCodeGuard ? Project Synopsis

Title: AutoCodeGuard ? Automotive Secure Code Review Assistant

Case S

Problem Statement

Modern automotive software systems ? from ADAS to infotainment ECUs ? are increasingly developed in high-level languages such as C, C++, and Python. Security vulnerabilities and coding defects in these systems can lead to safety failures, data breaches, and regulatory non-compliance. Development teams often lack immediate, accessible tooling that combines static analysis, AI-assisted explanation, and coding-guideline retrieval in a single review workflow. AutoCodeGuard addresses this gap with a lightweight, explainable prototype.

Objectives

1. Detect selected potentially unsafe code patterns using deterministic static analysis.

2. Prov

Scope

In scope:

• Pyth

Out of scope:

• Cer

Methodology

1. Static Analysis: Python AST visitor pattern detects syntactic anti-patterns (eval/exec, shell injection, bare except, mutable defaults, assert misuse, os.system). Regex rules detect credential leakage, weak hashes, SQL injection patterns, debug flags, TODO markers, and print statements.

2. LLM Integration: Source code is transmitted to Google Gemini only on user request, after displaying a privacy notice. A structured JSON prompt with a read-only reviewer persona elicits findings with severity, evidence, explanation, and recommendations.

Responses are validated before display.

3. Guideline Retrieval: A local JSON knowledge base of 16 guidelines (General, MISRA-inspired, Automotive, Security) is indexed with TF-IDF. Each finding triggers a cosine-similarity retrieval of the top-k most relevant guidelines.

4. Evaluation: A 12-case dataset (10 problematic + 2 safe) is used to evaluate the deterministic analyzer. Expected rule IDs per case allow computation of precision, recall, and F1.

Architecture

```
  User (Browser)
      ?
      ?
  Streamlit UI (app.py)
      ???? static_analyzer.py  ??? Python AST + Regex rules
      ???? gemini_integration.py ?? Google Gemini API (external)
      ???? guideline_retrieval.py ? TF-IDF over guidelines_kb.json
      ???? report_exporter.py  ??? JSON / CSV output
```

Technology Stack

Component	Technology
Language	Python 3.11+
UI	Streamlit
LLM	Google Gemini (gemini-flash-lite-latest, google-genai SDK)
Static Analysis	Python ast module + re
Guideline Retrieval	scikit-learn TF-IDF
Knowledge Base	Local JSON
Export	json, csv (stdlib)
Testing	pytest

Dataset

- Evaluation dataset: 12 hand-crafted test cases in Input_Data/evaluation_dataset.json

- K

catego

Evaluation Methodology

The deterministic static analyzer is evaluated by:

1. Run

LLM (Gemini) findings are advisory and cannot be evaluated with standard precision/recall.

Expected Outcomes

- Functional Streamlit application with full review workflow

- Stat

Limitations

- Heuristic rules produce potential findings, not confirmed vulnerabilities
- Ger

References

1. MISRA C:2012 ? Motor Industry Software Reliability Association (reference only)
2. ISO

Faculty Approval

Faculty Mentor Name:

Signature:

Date:

?? Note: This synopsis is a draft. Please verify against the prescribed PCCOE/Tata TechPulse template before final submission.